

Factory Reset Task - Reference

File: tasks/factory_reset.py | 235 lines

```
import time
import sys
import os

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from lib.device import Device
from lib.logger import Logger

SECTIONS = [
    "System",
    "About phone",
    "About device",
    "General management",
    "Additional Settings",
    "System & updates",
    "System settings",
    "Backup",
    "Reset",
    "Privacy",
]

RESET_LABELS = [
    "Reset phone",
    "Factory data reset",
    "Factory reset",
    "Reset options",
    "Back up and reset",
    "Backup & reset",
    "Reset",
]

ERASE_LABELS = [
    "Erase all data",
    "Erase everything",
    "Erase all",
]

CONFIRM_LABELS = [
    "Erase data",
    "Erase all data",
    "Erase everything",
    "Confirm",
    "Yes",
    "OK",
]

class FactoryResetTask:
    name = "factory-reset"
    description = "Factory reset an Android device and skip the setup wizard"

    def run(self, device=None, logger=None):
        if logger is None:
            logger = Logger(name="factory-reset", log_dir=".")
        if device is None:
            device = Device(log_dir=".")

        logger.info("=== Factory Reset Task ===")
        steps = 6

        logger.step(1, steps, "Connecting to device...")
        try:
            device.connect()
        except Exception as e:
            logger.error(f"Connection failed: {e}")
            return False
        logger.info(f"Device: {device.width}x{device.height}")

        logger.step(2, steps, "Navigating to factory reset...")
        if not self._navigate_to_reset(device, logger):
            logger.error("Could not find factory reset option.")
            device.screenshot("navigation_failed")
            return False

        logger.step(3, steps, "Tapping Erase all data...")
```

```

if not device.find_and_click_all(ERASE_LABELS, timeout=5):
    logger.error("Could not find Erase all data button.")
    device.screenshot("no_erase_button")
    return False
time.sleep(3)

logger.step(4, steps, "Triggering factory reset...")
try:
    self._trigger_reset(device, logger)
except Exception as e:
    logger.info(f"Device disconnected: {e}. Reset was triggered!")

logger.step(5, steps, "Waiting for device to reboot...")
logger.info("Device should disconnect now. Waiting for wipe + reboot...")
try:
    device.reconnect(timeout=300)
    logger.info("Device reconnected after reset")
except TimeoutError:
    logger.error("Device did not reconnect within 5 minutes")
    return False

logger.step(6, steps, "Skipping setup wizard...")
self._skip_setup(device, logger)

device.go_home()
logger.done("Factory reset completed. Device is on home screen.")
return True

def _navigate_to_reset(self, d, log):
    d.wake()
    time.sleep(1)

    # Force-stop to clear back stack ? ensures we always start fresh
    log.info("Force-stopping Settings...")
    d.d.shell(["am", "force-stop", "com.android.settings"])
    time.sleep(2)

    log.info("Opening fresh Settings...")
    d.d.app_start("com.android.settings")
    time.sleep(2)

    d.d.shell(["wm", "dismiss-keyguard"])
    d.d.screen_on()
    time.sleep(3)

    # Tap the search bar
    log.info("Tapping Search bar...")
    d.d(text="Search").click()
    time.sleep(2)

    # Type "reset phone"
    log.info('Typing "reset phone"...')
    d.d.send_keys("reset phone")
    time.sleep(3)

    # Tap the first search result containing reset-related text
    log.info("Opening search result...")
    for label in RESET_LABELS:
        el = d.d(text=label)
        if el.wait(timeout=2):
            el.click()
            time.sleep(3)
            log.info(f"Opened: {label}")
            return True

    log.warning("Could not find reset option via search.")
    d.dump_ui()
    return False

def _trigger_reset(self, d, log):
    log.info("Confirming factory reset...")
    for attempt in range(3):
        found = d.find_and_click_all(CONFIRM_LABELS, timeout=4)
        if found:
            log.info(f"Tapped confirmation ({attempt + 1}/3)")
            time.sleep(3)
        else:
            log.info(f"No button found on attempt {attempt + 1}, tapping bottom-center...")
            d.d.click(d.width // 2, int(d.height * 0.84))
            time.sleep(3)

    log.info("Factory reset command sent. Device should reboot.")

```

```

def _skip_setup(self, d, log):
    log.info("Waiting for setup wizard...")
    time.sleep(5)

    log.info("Trying fast path (direct flags)...")
    d.d.shell(["settings", "put", "global", "device_provisioned", "1"])
    d.d.shell(["settings", "put", "secure", "user_setup_complete", "1"])
    d.d.shell(["settings", "put", "global", "setup_wizard_has_run", "1"])
    time.sleep(2)

    if self._is_home(d):
        log.info("Fast path worked! Already on home screen.")
        return

    log.info("Walking through setup wizard screens...")
    deadline = time.time() + 600
    step = 1
    setup_actions = [
        ("Start", d.find_and_click, ["Start"]),
        ("Continue", d.find_and_click, ["Continue"]),
        ("Skip", d.find_and_click, ["Skip"]),
        ("Not now", d.find_and_click, ["Not now"]),
        ("Accept", d.find_and_click, ["Accept"]),
        ("Agree", d.find_and_click, ["Agree"]),
        ("Next", d.find_and_click, ["Next"]),
        ("Done", d.find_and_click, ["Done"]),
        ("Get started", d.find_and_click, ["Get started"]),
        ("Don't copy", d.find_and_click, ["Don't copy"]),
        ("Copy", d.find_and_click, ["Copy"]),
        ("Set up later", d.find_and_click, ["Set up later"]),
        ("Remind me later", d.find_and_click, ["Remind me later"]),
        ("Skip anyway", d.find_and_click, ["Skip anyway"]),
    ]

    while time.time() < deadline:
        focus = d.current_focus().lower()
        if self._is_home(d):
            log.info("Home screen detected. Setup complete!")
            return

        acted = False
        for label, action, args in setup_actions:
            if action(*args, timeout=1):
                log.step(step, 99, f"Tapped '{label}'")
                step += 1
                acted = True
                time.sleep(2)
                break

        if not acted:
            log.info(f"Unknown screen, waiting... ({focus[:60]}")
            time.sleep(5)
            step += 1

        if step > 120:
            log.error("Too many attempts. Giving up.")
            d.screenshot("setup_timeout")
            break

    log.info("Setup wizard navigation finished.")

def _is_home(self, d):
    focus = d.current_focus().lower()
    return any(x in focus for x in ["launcher", "home", "desktop"])

def _get_clickable_texts(self, xml):
    import re
    texts = re.findall(r'text="([^\"]+)"', xml)
    seen = set()
    result = []
    for t in texts:
        t = t.strip()
        if t and len(t) > 1 and t not in seen:
            seen.add(t)
            result.append(t)
    return result

```